Innovative Evaluation Practices for BCSE308P Computer Networks Lab

Fall Semester 2026-27

CCSDS INSPIRED SOFTWARE TELEMETRY RECEIVER AND GROUND STATION

24BCE2055 JAHNAVI SINGH

B.Tech.

in

Computer Science and Engineering

School of Computer Science and Engineering



October 2026

ABSTRACT

A telemetry receiver must identify frame boundaries before it can interpret the measurements carried by a stream. This becomes difficult when a marker crosses packet boundaries or an inserted or deleted bit shifts every later frame. The project addresses this problem through a software simulation of a spacecraft transmitter and a ground station connected by UDP.

The transmitter generates custom 256-byte frame bodies, protects them with CRC-16, and places the 32-bit marker 0x1ACFFC1D before each body. Controlled faults are introduced before the stream is divided into small datagrams. The receiver retains incomplete data in a persistent bit buffer and follows four states: HUNT, LOCK, VERIFY and EXTRACT. It searches at every bit position. When a candidate fails validation, searching resumes one bit after that candidate's marker, allowing later intact frames to be recovered without restarting the receiver.

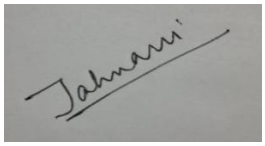
Evaluation compared the receiver with a persistent byte-aligned parser across ten paired packetization seeds. Both accepted all 100 clean frames. After a single insertion or deletion in frame 20, the proposed receiver accepted 99 frames, while the baseline accepted 20. The proposed receiver also recovered all frames following a three-bit prefix and an embedded false marker. The current implementation passed 185 automated tests, type checking and a production build.

The results support the value of bit-level reacquisition for the tested logical streams. The system remains a local teaching model: its frame body is custom, CRC detects rather than repairs damage, and the receiver does not restore UDP ordering.

CERTIFICATE

This is to certify that the project report titled "CCSDS Inspired Software Telemetry Receiver and Ground Station" is a bonafide record of the work done by JAHNAVI SINGH, Registration Number 24BCE2055, in partial fulfilment of the requirements of the Innovative Evaluation Practices for BCSE308P - Computer Networks Lab during the Fall Semester 2026-2027.

JAHNAVI SINGH
24BCE2055

Signature ____  ___

Name and Signature of the Student

TABLE OF CONTENTS

ABSTRACT **2**

CERTIFICATE **3**

1 INTRODUCTION **6**

1.1 Background and motivation 6

1.2 Problem statement 6

1.3 Objectives 6

1.4 Scope and limitations 6

1.5 Mapping to course modules and the lab experiment 7

2 LITERATURE SURVEY **8**

2.1 Overview of the existing protocol or scheme 8

2.2 Review of related work 8

2.3 Research gap and justification of novelty 8

3 PROPOSED SYSTEM **9**

3.1 System architecture or topology 9

3.2 Baseline mechanism description 10

3.3 Proposed mechanism algorithm pseudocode and flowchart 10

3.4 Novelty statement 11

3.5 Assumptions and design decisions 12

4 IMPLEMENTATION **13**

4.1 Tools platform versions and hardware 13

4.2 Simulation or testbed setup 13

4.3 Code structure and explanation of key modules 14

4.4 Challenges faced and how they were solved 15

5 EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION **16**

5.1 Performance metrics and why they were chosen 16

5.2 Scenarios and parameter values 16

5.3 Results tables and graphs 17
5.4 Analysis and discussion 19
5.5 Critical evaluation 19
6 CONCLUSION AND FUTURE WORK 21
6.1 Summary of work and key findings 21
6.2 Limitations 21
6.3 Future enhancements 21
7 REFERENCES 22
APPENDIX A IMPORTANT SOURCE CODE AND CONFIGURATION 23
A.1 CRC implementation 23
A.2 Candidate rejection and reacquisition 23
A.3 Experimental baseline 24
A.4 Commands for verification and demonstration 24
APPENDIX B ADDITIONAL EVIDENCE AND RAW RESULTS 25
B.1 Recorded localhost dashboard evidence 25
B.2 Raw accepted-frame counts 27
B.3 Raw workload execution times 27
B.4 Verification record 28
B.5 Additional interface screenshots 29

1 INTRODUCTION

1.1 Background and motivation

Telemetry turns measurements from a remote system into information that can be inspected at a receiving station. Before the receiver can decode a temperature or a battery voltage, it must determine where a frame begins and whether the received bytes can be accepted. A small error in boundary detection can make every later field appear meaningless even when most of the stream is intact.

This project concentrates on that receiving task. A software spacecraft generates repeatable measurements, and a local ground station reconstructs them from a sequence of UDP payloads. Small, deliberately chosen datagrams make frame boundaries different from network message boundaries. Bit offsets, corruption and bit slips then expose the assumptions made by the parser.

1.2 Problem statement

UDP provides a datagram service without guaranteed delivery or duplicate protection [1]. It does not divide a delivered application datagram into separate message callbacks. Here, the application first constructs a logical telemetry stream and then deliberately sends pieces of that stream in different datagrams. The receiver therefore has to retain partial markers and partial frames across callbacks.

A byte-aligned marker search is sufficient only while the logical stream remains aligned to the received bytes. Adding or removing one bit moves subsequent markers to a different position within each byte. The problem is to reject the damaged candidate, search the retained stream at the new alignment, and resume valid decoding without being given the fault location.

1.3 Objectives

The primary objective is to recover valid telemetry independently of UDP datagram boundaries. This requires persistent buffering, marker detection at alignments 0 through 7, an explicit synchronization state machine, and CRC-controlled acceptance. A failed candidate must lead to renewed searching rather than a process restart. The implementation must also recover when an intact marker is located inside a previously rejected candidate.

The supporting objectives are repeatable fault simulation, bounded receive storage, meaningful frame-sequence metrics, and a browser view that makes the receiver's decisions visible. Automated tests are used to check exact decoded values, stream positions and recovery behaviour.

1.4 Scope and limitations

The system runs on one computer using localhost UDP and a WebSocket link to the browser. It uses the real CCSDS Attached Sync Marker (ASM), but the 256-byte body is a project-defined teaching format. It does not implement the complete CCSDS TM Transfer Frame, channel coding, or a physical radio link [2], [3].

The receiver processes UDP payloads in arrival order. It does not retransmit missing data, reconstruct reordered datagrams, or correct corrupted frames. Bit insertion and deletion are applied to the logical stream before packetization; they do not represent behavior directly exposed by an ordinary UDP socket. Database storage, authentication and cloud deployment are outside the project scope.

1.5 Mapping to course modules and the lab experiment

The implementation connects protocol concepts to observable program behaviour. Table 1 maps the main course areas to the corresponding laboratory activity.

Course area	Project activity	Observable evidence
Transport layer and sockets	Send and receive UDP datagrams through node:dgram	Datagram and payload-byte counters
Framing and synchronization	Reassemble a logical stream and search for the ASM	Markers and bodies spanning multiple arrivals
Error detection	Calculate and compare CRC-16	Rejected candidate with stored and calculated CRC
Protocol state machines	Execute HUNT, LOCK, VERIFY and EXTRACT	Transition traces and exact state tests
Performance evaluation	Compare parsers with the same seeded input	Repeated-run recovery and execution-time results
Client and server communication	Publish receiver observations over WebSocket	Live dashboard and guided recordings

Table 1 Mapping of course concepts to the laboratory implementation

2 LITERATURE SURVEY

2.1 Overview of the existing protocol or scheme

RFC 768 defines the UDP transport used here [1]. CCSDS synchronization material specifies the selected ASM [2], while the TM Space Data Link Protocol defines the fuller transfer-frame format that this teaching body does not implement [3]. These standards establish the project's technical starting point.

PPP in HDLC-like framing examines a bit stream for flags and uses stuffing and a frame check sequence [4]. Although its framing differs from this project, it is a useful example of explicit boundary detection, transparency and validation.

2.2 Review of related work

Table 2 compares the relevant standards and implementation references. These sources specify mechanisms and interfaces; they are not performance studies against the receiver implemented here.

No	Authors and year	Technique	Metrics and tool	Merits	Gaps for this project
1	J. Postel, 1980 [1]	UDP datagram transport	Message delivery; socket interface	Small transport mechanism	No application frame reconstruction or ordering repair
2	CCSDS, 2023 [2]	ASM and telemetry synchronization	Marker pattern; channel-coding rules	Established synchronization reference	Full channel-coding scope exceeds this local model
3	CCSDS, 2021 [3]	TM transfer-frame protocol	Frame structure; service and error-control rules	Defines actual TM frame requirements	Custom body here does not implement the complete protocol
4	W. Simpson, 1994 [4]	HDLC-like PPP framing	Flag detection; stuffing; frame check sequence	Explicit framing and transparency rules	Different delimiter, length and stuffing conventions
5	Node.js contributors, 2025 [5]	UDP socket API	Buffer message events and socket errors	Direct implementation support	Application must supply its own stream framing

Table 2 Related standards and implementation references

2.3 Research gap and justification of novelty

The project addresses reconstruction between received datagrams and validated telemetry. A persistent byte parser still misses markers after the alignment shifts. Combining bit storage, every-bit searching and validation-driven reacquisition removes that assumption. The experimental control measures this specific benefit; it does not establish a new general synchronization algorithm or superiority over a complete CCSDS receiver.

3 PROPOSED SYSTEM

3.1 System architecture or topology

The transmitter and receiver are separate processes. The transmitter builds a complete frame before the simulator applies a fault. The packetizer then selects datagram boundaries from the resulting stream. On reception, the UDP callback passes each payload unchanged to the same Synchronizer instance. No target frame, fault position, or alignment hint is supplied to that receiver.

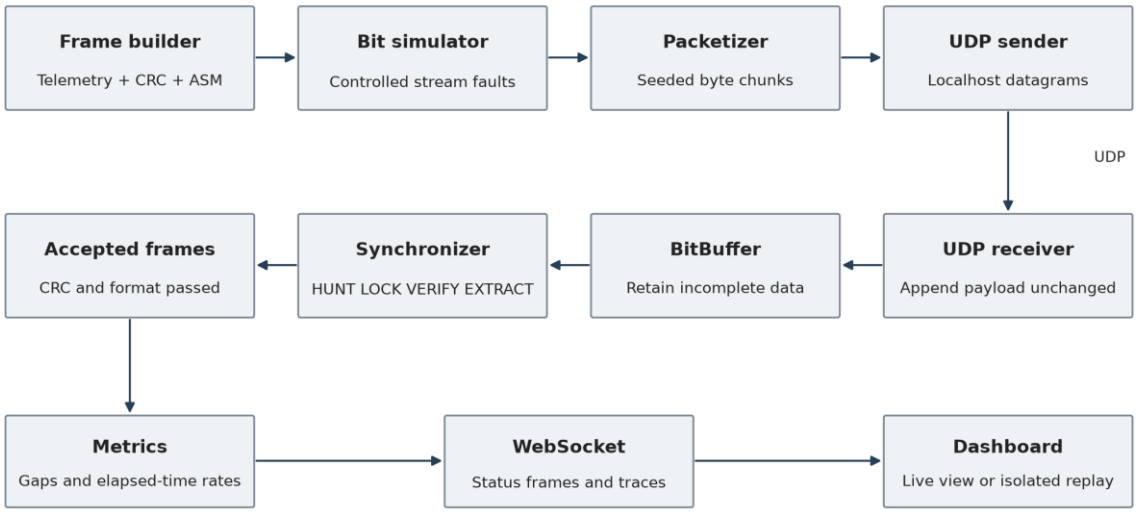


Figure 1 Data path from generated telemetry to the browser observer

The browser receives accepted frames, status snapshots and bounded traces. Its guided mode requests a six-frame recording generated by an isolated instance of the same production engine. Guided chunks are processed in memory and do not reset the live receiver. This distinction matters during demonstrations: live mode shows network arrivals, while guided mode makes fast state transitions available one step at a time.

Each transmission unit consists of the external four-byte ASM and the body shown in Figure 2. All multibyte integers use big-endian order, and bits are processed most significant first.

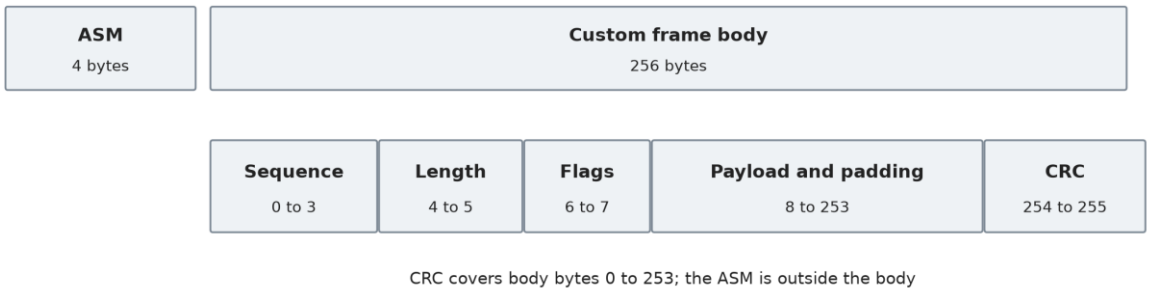


Figure 2 The external marker and custom 256-byte frame body

Body bytes	Field	Size and meaning
0 to 3	sequence	Unsigned 32-bit frame counter
4 to 5	payloadLength	Unsigned 16-bit value from 0 to 246
6 to 7	flags	Unsigned 16-bit reserved field; default zero
8 to 253	payload	246-byte capacity, with unused bytes zero-filled
254 to 255	crc16	CRC over body bytes 0 to 253

Table 3 Field positions in the custom frame body

The default payload contains 12 meaningful bytes. Temperature occupies two signed bytes in centi-degrees Celsius, battery voltage occupies two unsigned bytes in millivolts, and altitude and uptime each occupy four unsigned bytes. Uptime is generated from the sequence number; it is not elapsed host time. A valid payload shorter than 12 bytes is accepted without inventing telemetry values.

3.2 Baseline mechanism description

The baseline is a small experimental byte-aligned parser constructed for this report. It retains data between chunks, uses the same BitBuffer and frame decoder, waits for a complete candidate, and rejects candidates with an invalid CRC or payload length. It searches for the marker at offsets 0, 8, 16 and so on. After a failed candidate it advances eight bits before searching again.

Keeping persistence and validation in the baseline makes the comparison more informative than treating every datagram as a complete frame. Both receivers must handle arbitrary chunk boundaries; the tested difference is whether the search can leave the original byte alignment. The baseline is an educational control, not a complete PPP or CCSDS receiver. Its source is separate from the production code.

3.3 Proposed mechanism algorithm pseudocode and flowchart

HUNT searches each available 32-bit window for the ASM. Once a marker is found, LOCK preserves that candidate until the buffer contains 2,080 bits from its start: 32 marker bits and 2,048 body bits. VERIFY rebuilds the body as a byte-aligned Buffer and calls the shared decoder. CRC and format must both pass before EXTRACT emits the frame.

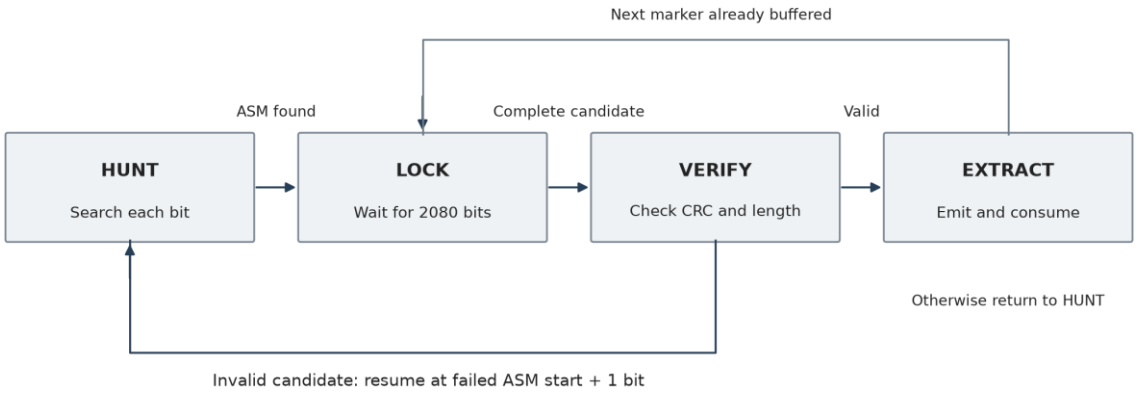


Figure 3 Receiver states and validation-driven recovery

The CRC uses polynomial 0x1021, initial value 0xFFFF, no input or output reflection, and final XOR 0x0000. The known input 123456789 produces 0x29B1. CRC covers the body up to byte 253, including padding, while the ASM and stored CRC are excluded. A mismatch rejects the candidate; it does not repair the damaged bytes.

```
On each received datagram:
    count its bytes and append them in bounded batches
    while a state transition can be completed:
        HUNT:
            compare every available 32-bit window with ASM
            retain up to 31 trailing bits if no marker matches
            otherwise position the head at the marker and LOCK
        LOCK:
            wait if fewer than 2080 bits are available
            otherwise move to VERIFY
        VERIFY:
            extract 256 body bytes and check CRC and length
            if invalid, discard one bit from the marker start
            and return to HUNT
            otherwise move to EXTRACT
        EXTRACT:
            emit the accepted frame and consume 2080 bits
            LOCK if the next marker is already at the head
            otherwise return to HUNT
```

The one-bit restart is necessary for a false lock. In the protected test case, a false marker starts at bit 0 and a true marker starts at bit 35, inside the rejected candidate. Returning to bit 1 leaves the true marker available. Discarding the whole 2,080-bit candidate would skip it.

3.4 Novelty statement

Relative to the baseline, the proposed receiver searches at all eight bit alignments and restarts one bit after a rejected marker. It also exposes the HUNT, LOCK, VERIFY and EXTRACT transitions directly. The measured benefit is recovery after changed alignment, rather than improved clean-stream delivery. Both mechanisms recover the same clean frames and reject the deliberately flipped frame.

UDP, ASM patterns, CRC, state machines and buffers remain established components. The originality of the course project lies in implementing their interaction transparently, preserving candidate context across fragments, and testing the failure cases that a byte-aligned parser cannot resolve.

3.5 Assumptions and design decisions

Frame length and field positions are fixed for the duration of a stream. The receiver accepts datagram payloads in arrival order and has no transport-level datagram sequence field. Sequence gaps therefore refer to missing valid-frame observations. The first accepted frame establishes the sequence baseline; losses before that observation cannot be inferred.

The receiver retains at most 31 search-tail bits in HUNT and preserves a partial candidate in LOCK. Its backing buffer is capped at 64 KiB, and input is processed in 4 KiB batches. Physical compaction does not change the logical stream position. The memory bound applies to receive storage, not to the entire Node.js process.

CRC failure takes precedence if CRC and structure both fail. A CRC-valid length above 246 increments formatFailures. resyncCount increments when another candidate is found after a failed one; the new candidate may still fail. These counting rules are explicit so that a counter is not mistaken for proof that all missing data was recovered.

4 IMPLEMENTATION

4.1 Tools platform versions and hardware

The application is a single TypeScript package. Node's built-in dgram module provides UDP, Buffer handles binary data, and ws is the only external runtime dependency. The dashboard uses vanilla TypeScript, HTML and CSS with Vite. Table 4 records the implementation and evaluation environment.

Component	Version or configuration
Application runtime and package manager	Node.js 22.19.0; npm 10.9.3
Language and execution helper	TypeScript 5.9.2; tsx 4.20.5
Browser bridge and UI tooling	ws 8.18.3; Vite 7.1.3
Automated tests	node:test and node:assert
Evaluation runtime	Node.js 24.19.0 for both comparison parsers
Processor	13th Gen Intel(R) Core(TM) i5-13420H
Memory and operating system	15.7 GiB RAM; Windows build 10.0.26200
Network topology	One host, IPv4 loopback; no RF or external network

Table 4 Software and hardware environment

The project verification used its pinned Node 22.19.0 environment. The separate comparison harness ran both parsers under the same available Node 24.19.0 runtime. Its timings describe that harness and runtime, not a benchmark of the pinned application deployment.

4.2 Simulation or testbed setup

The live testbed has a UDP transmitter sending to 127.0.0.1:5000, a receiver publishing to ws://127.0.0.1:8080, and a Vite page at http://127.0.0.1:5173. The receiver remains active without a browser. WebSocket is an observation channel and is not part of the simulated spacecraft downlink [6].

Parameter	Value	Reason
Frame body and complete unit	256 and 260 bytes	Fixed extraction size
Default generation rate	5 frames per second	Readable local demonstration
Datagram payload range	1 to 96 bytes	Frequent splits within markers and frames
Packetization seed	42 by default	Repeatable chunk boundaries

Parameter	Value	Reason
Default fault location	Frame index 5; body bit 80	Single reproducible fault after CRC
Comparison fault location	Frame index 20; body bit 80	Observe both pre-fault and post-fault behavior
Prefix offset	3 bits in the main comparison	Expose non-byte-aligned acquisition
Receive backing cap and batch	64 KiB and 4 KiB	Bound receive storage

Table 5 Testbed configuration and parameter choices

For a live demonstration, the receiver is started first, followed by the transmitter and the dashboard. The following commands are executed in separate terminals from the project directory.

```
npm run receiver
npm run transmitter -- --mode clean
npm run ui
```

The simulator supports clean, bit-offset, bit-flip, bit-insert and bit-delete; bit-slip is an alias for insertion. Faults are applied after CRC generation and before packetization. The simulator retains unfinished bits between frames, and the packetizer retains unfinished datagrams. Only final shutdown pads a partial simulator byte and flushes the remaining datagram.

4.3 Code structure and explanation of key modules

The shared directory defines the wire constants, CRC, frame codec, configuration and observation types. The encoder validates the sequence, field ranges and payload capacity before writing. The decoder checks the stored CRC and declared length, copies the meaningful payload, and decodes telemetry only when at least 12 valid payload bytes exist.

On the transmitter side, FrameBuilder increments a uint32 sequence and produces deterministic sensor values. BitstreamSimulator walks the transmitted bits in MSB-first order and applies a selected fault at an explicit position. Packetizer uses a persistent Mulberry32 state to choose chunk sizes independently of frame boundaries. The UDP sender transmits those chunks without acknowledgements or retries.

On the receiver side, BitBuffer maintains owned storage and a head offset measured in bits. Synchronizer owns the four states and protocol counters. The UDP wrapper owns datagram and byte counts. Metrics tracks forward sequence advances and uses elapsed monotonic time for rates. The distinction prevents transport and protocol counters from being confused.

The WebSocket bridge emits receiver.status at most four times per second and frame.valid for every accepted frame. The current teaching interface additionally uses receiver.trace and lesson.recording; lesson.start is the browser request for an isolated recording. Trace previews and transition lists are bounded. The browser can replay a lesson, inspect captured marker

windows, compare stored and calculated CRC values, or freeze live visuals while reception continues.

4.4 Challenges faced and how they were solved

The first design difficulty is preserving incomplete information. A marker can end in one datagram and continue in another, so clearing the buffer after each callback is incorrect. Persistent storage and the 31-bit search tail solve that boundary case. LOCK similarly preserves an incomplete body until enough input has arrived.

A second difficulty is rejecting a candidate without skipping useful data. A false marker may be followed by a real marker before the candidate's expected end. Advancing one bit after rejection preserves that possibility. Tests assert the exact restart position rather than checking only that a later frame eventually appears.

The alignment displayed to the user must survive storage compaction. The implementation therefore records absolute logical bit position separately from buffer indices. Insertion changes later alignment from 0 to 1, and deletion changes it from 0 to 7. The UI reports current candidate alignment separately from the latest accepted frame's alignment.

Brief VERIFY and EXTRACT transitions can occur between periodic status messages. Bounded per-arrival traces and guided recordings make those transitions inspectable. The guided recording is kept separate from live state so that teaching controls do not influence synchronization decisions.

5 EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

5.1 Performance metrics and why they were chosen

The main outcome is accepted-frame ratio: accepted valid frames divided by generated frames, multiplied by 100. It directly measures how much of the offered stream becomes usable telemetry. The denominator includes the deliberately damaged frame, so 99 accepted frames out of 100 represents successful rejection and recovery in the single-fault cases.

Recovery span is the distance in stream bits from the failed frame's marker to the next accepted marker. It avoids converting an input position into an unsupported network delay claim. CRC failures confirm candidate rejection but are not a count of every corrupt bit or lost datagram. An offset stream can produce zero CRC failures simply because a byte-aligned parser never finds a candidate.

Execution time measures only feeding prebuilt chunks into a parser and collecting its output. Stream generation, fault injection, packetization, sockets, deliberate transmission delays and browser rendering are outside the timed interval. Accepted ratio and time are reported together so that a parser that rejects or misses most frames is not described as better merely because it finishes sooner.

For ten observations x1 to x10, the mean is their sum divided by ten. Sample standard deviation is the square root of the sum of squared deviations from the mean divided by nine. Relative improvement is 100(P - B)/B, where P and B are proposed and baseline means. When B is zero, relative improvement is undefined; the absolute percentage-point gain is stated instead.

5.2 Scenarios and parameter values

The comparison was executed on 20 October 2026. Each main scenario generated sequences 0 through 99. Ten runs used packetization seeds 42 through 51, with the same transformed stream and identical chunk boundaries supplied to both receivers within each pair. Three untimed warm-up passes preceded each scenario, and parser order alternated between runs. Every run asserted the expected accepted-frame count.

ID	Scenario	Changed condition
S1	Clean	No prefix or corruption
S2	Offset 3	Three alternating prefix bits
S3	Bit flip	One body-bit flip in frame 20
S4	Bit insertion	One inserted bit in frame 20
S5	Bit deletion	One deleted bit in frame 20
S6	Embedded true marker	False ASM at bit 0; true stream starts at bit 35
L1 to L3	Clean workload sweep	100, 500 and 1000 generated frames

Table 6 Scenarios used in the paired comparison

All comparison chunks were generated with a minimum of one and a maximum of 96 bytes. L1, L2 and L3 vary finite workload size, not offered network rate. They are low, medium and high parser workloads within this experiment; no congestion or saturation claim is made. Fresh parser instances were used for each measured run.

The repeated comparison is an in-memory evaluation of framing behavior. Real localhost UDP and WebSocket operation are covered independently by the automated suite and the archived live demonstrations. A browser screenshot cannot establish a ten-run average, and an in-memory result cannot establish network delivery probability.

5.3 Results tables and graphs

Table 7 gives the mean accepted-frame ratio and sample SD across the ten paired runs. Zero SD is expected here: different packetization boundaries did not alter the deterministic acceptance outcome in any tested case. It does not establish a probability of perfect recovery on arbitrary traffic.

ID	Scenario	Baseline % mean ± SD	Proposed % mean ± SD	Relative gain
S1	Clean	100 ± 0	100 ± 0	0%
S2	Offset 3	0 ± 0	100 ± 0	N/A
S3	Bit flip	99 ± 0	99 ± 0	0%
S4	Bit insertion	20 ± 0	99 ± 0	395%
S5	Bit deletion	20 ± 0	99 ± 0	395%
S6	False marker at bit 0	0 ± 0	100 ± 0	N/A

Table 7 Accepted-frame ratios for 100-frame runs with ten seeds

For S2 and S6, the baseline mean is zero, so relative gain is not defined. The proposed receiver gains 100 percentage points in each case. For S4 and S5, the gain is 79 percentage points, equivalent to 395% relative to the baseline's 20% acceptance. S1 and S3 have equal acceptance in the two mechanisms.

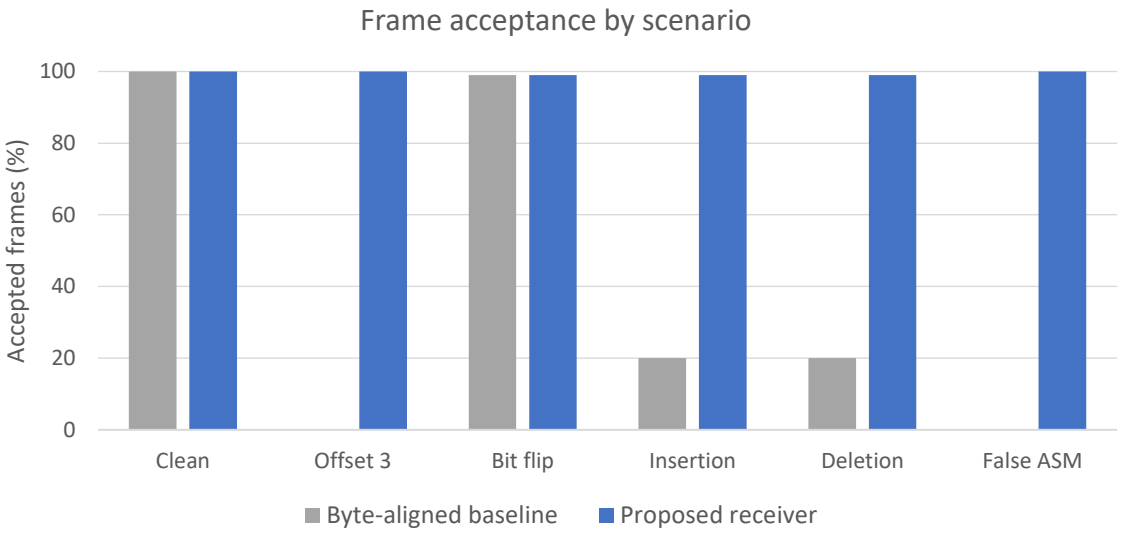


Figure 4 Accepted-frame ratio for the baseline and proposed receiver

In S4 and S5 the baseline accepted sequences 0 to 19 and found no later intact marker at a byte boundary. The proposed receiver rejected sequence 20 and resumed at 21. Its next marker occurred 2,081 bits after the failed marker for insertion and 2,079 bits after it for deletion. A simple flip retained the original alignment and gave a 2,080-bit span in both parsers.

Generated frames	Baseline time ms mean ± SD	Proposed time ms mean ± SD	Proposed acceptance
100	1.46 ± 0.27	1.47 ± 0.42	100 / 100
500	7.86 ± 2.31	8.30 ± 2.42	500 / 500
1000	15.72 ± 4.74	16.04 ± 3.83	1000 / 1000

Table 8 Execution time for increasing clean parser workloads

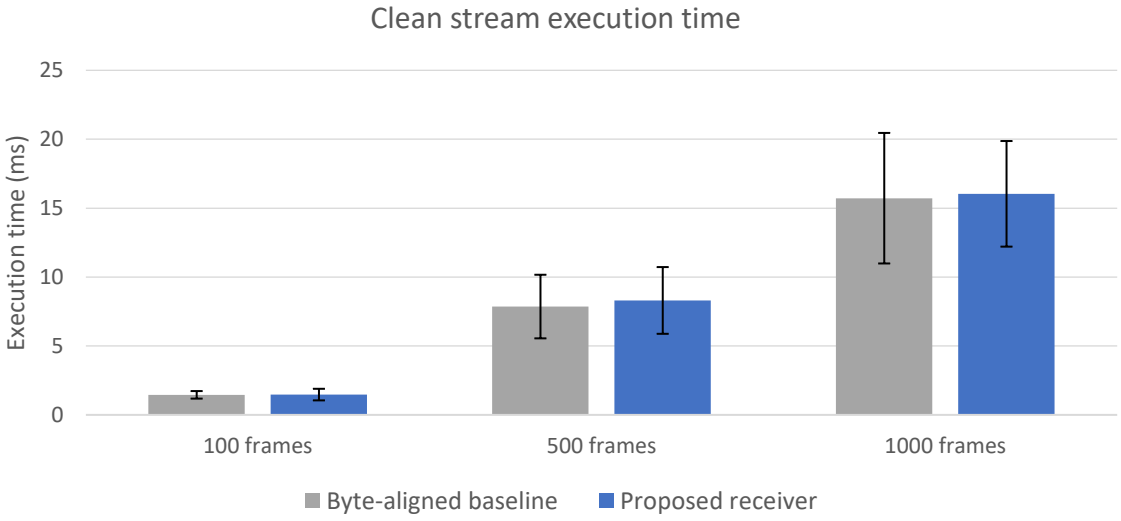


Figure 5 Clean-stream parser time with sample standard deviation

Both parsers accepted every clean frame at all three workload sizes. Their measured times grew with the amount of input processed. The small mean differences and overlapping variation do not justify a claim that either parser is generally faster. The purpose of the added bit search is to retain recovery capability when byte alignment changes.

5.4 Analysis and discussion

S1 separates fragmentation from corruption. Both receivers keep partial data, so neither loses clean frames merely because the packetizer chooses a new boundary. This result supports persistence as a necessary design property, but it does not distinguish the two marker-search strategies.

S2 isolates the alignment assumption. The three-bit prefix does not damage the following frames, yet none of their markers falls on a received byte boundary. The byte-aligned parser therefore accepts nothing. The proposed receiver acquires alignment 3 and accepts all 100 frames without requiring a CRC failure first.

S3 shows a case where the baseline is adequate. A single flipped body bit invalidates one candidate while leaving later markers byte aligned. Both parsers reject one frame and recover 99. S4 and S5 create a different condition: one changed bit shifts all later boundaries. The proposed receiver reacquires the next intact marker at alignment 1 or 7, while the baseline remains unable to recognize it.

S6 checks the rejected-candidate rule. The first marker is false, but a true marker lies at bit 35 within its candidate window. The proposed receiver preserves the remaining bits after rejection and finds that true start. Together with the exact-position unit test, this provides stronger evidence than a demonstration that only places a valid marker after the whole rejected frame.

The principal trade-off is additional bit-level work when alignment is unknown or noise must be scanned. A parser that assumes byte alignment examines fewer candidate positions. On an unsupported stream it may appear quick because it does not decode the valid frames it has missed. That apparent saving must be read alongside its accepted-frame ratio.

5.5 Critical evaluation

The implementation’s strengths are explicit state transitions, reproducible faults, independent protocol tests and bounded receive storage. It protects incomplete candidates and can recover after false locks and changed alignment. Its standard components and small module boundaries make the behavior feasible to explain during a viva.

Recovery still requires a later intact marker and enough following body bits to validate a frame. A damaged marker can prevent that frame from becoming a candidate at all. Persistent corruption, an incomplete final stream, or an error pattern that passes the finite CRC can defeat acceptance decisions. No retransmission or correction mechanism restores the rejected data.

The transport remains unreliable. Datagram loss, duplication or reordering changes the reconstructed stream, and this receiver does not restore the original order. Frame-sequence gaps cannot identify the number of missing datagrams. Since the first valid sequence establishes the

baseline, a receiver that never reacquires cannot count all subsequent missing frames through that metric alone.

The comparison used one deterministic telemetry generator, selected fault locations and ten packetization seeds. The workload sweep did not measure network saturation, RF conditions or long-duration process memory. A production receiver would need a much wider evaluation and an agreed protocol specification.

The current implementation also has modest practical limits. Oversized configured UDP chunks are not rejected early enough, changing WS_PORT requires updating the browser endpoint, and a reconnect while visuals are frozen needs clearer UI handling. These are maintenance items; they do not explain the recovery differences measured in the default local configuration.

6 CONCLUSION AND FUTURE WORK

6.1 Summary of work and key findings

The project implements a complete local telemetry path from synthetic measurements to validated browser observations. Its receiver reconstructs frames across arbitrary UDP payload boundaries and accepts data only after CRC and structural checks. The central recovery rule preserves the stream after a failed candidate and searches again at the next bit.

Across ten paired seeds, both parsers accepted all 100 clean frames and 99 frames after a single bit flip. After insertion or deletion in frame 20, the proposed receiver accepted 99 frames while the byte-aligned baseline accepted 20. A three-bit prefix and the embedded true-marker case each produced 100 accepted frames in the proposed receiver and zero in the baseline. These outcomes identify changed alignment as the condition where the proposed mechanism adds value.

The current project passed 185 automated tests, strict type checking and the UI production build. Those checks include real UDP reconstruction, all eight alignments, both slip directions, CRC and format rejection, sequence wraparound, bounded noise handling and lesson isolation. The result is a working laboratory demonstration of framing and recovery under the stated conditions.

6.2 Limitations

The frame body is custom, the telemetry values are generated, and faults are injected into a logical software stream. The receiver does not implement complete CCSDS conformance, RF processing, error correction, retransmission or reorder repair. Its CRC cannot prove that every accepted candidate is authentic, and the measured parser timings should not be read as an end-to-end network benchmark.

6.3 Future enhancements

The next useful extensions would be deterministic datagram drop, duplicate and adjacent-reorder modes, followed by tests that distinguish frame loss from transport effects. A broader fault campaign could vary body position, multiple simultaneous faults, marker damage and long noise bursts. These cases would strengthen the recovery evidence without requiring a larger application framework.

Practical maintenance should first add early UDP chunk-size validation, a configurable browser endpoint and browser regression checks for reconnect and replay controls. Persistent telemetry history or forward error correction would be separate extensions with their own design and evaluation requirements.

7 REFERENCES

[1] J. Postel, “User Datagram Protocol,” RFC 768, Aug. 1980. doi: 10.17487/RFC0768. [Online]. Available: https://www.rfc-editor.org/rfc/rfc768.html. Accessed: Oct. 20, 2026.

[2] Consultative Committee for Space Data Systems, TM Synchronization and Channel Coding, CCSDS 131.0-B-5, Sep. 2023, sec. 9. [Online]. Available: https://ccsds.org/Pubs/131x0b5.pdf. Accessed: Oct. 20, 2026. This is the issue used for the marker reference, rather than a claim about the latest issue.

[3] Consultative Committee for Space Data Systems, TM Space Data Link Protocol, CCSDS 132.0-B-3, Oct. 2021. [Online]. Available: https://ccsds.org/Pubs/132x0b3.pdf. Accessed: Oct. 20, 2026.

[4] W. Simpson, Ed., “PPP in HDLC-like Framing,” RFC 1662, Jul. 1994. doi: 10.17487/RFC1662. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1662.html. Accessed: Oct. 20, 2026.

[5] Node.js contributors, “UDP/datagram sockets,” Node.js v22.19.0 Documentation, 2025. [Online]. Available: https://nodejs.org/download/release/v22.19.0/docs/api/dgram.html. Accessed: Oct. 20, 2026.

[6] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Dec. 2011. doi: 10.17487/RFC6455. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6455.html. Accessed: Oct. 20, 2026.

[7] Telemetry Ground Station project repository, product requirements, source code and automated tests, local version inspected Oct. 20, 2026.

[8] Telemetry Ground Station project, “Final PRD Audit,” Oct. 8, 2026, and archived localhost demonstration records in dist/manual-round7.

[9] Telemetry receiver comparison experiment, ten paired seeds per scenario, raw results and parser timing observations, Oct. 20, 2026. Evaluation method and selected raw records appear in Section 5 and Appendix B.

APPENDIX A IMPORTANT SOURCE CODE AND CONFIGURATION

A.1 CRC implementation

The following excerpt is from src/shared/crc16.ts. The same implementation is used when building and validating frame bodies.

```
export function crc16Ccitt(data: Buffer): number {
  let crc = CRC_INITIAL_VALUE;
  for (const byte of data) {
    crc ^= byte << 8;
    for (let bit = 0; bit < 8; bit++) {
      crc = ((crc & 0x8000) !== 0
        ? (crc << 1) ^ CRC_POLYNOMIAL : crc << 1) & 0xffff;
    }
  }
  return (crc ^ CRC_FINAL_XOR) & 0xffff;
}
```

A.2 Candidate rejection and reacquisition

This excerpt from src/receiver/synchronizer.ts shows the rejection decision. The candidate marker is already at the buffer head, so discard(1) implements the failed-marker-plus-one rule.

```
if (!decoded.crcValid || !decoded.formatValid) {
  const reason = !decoded.crcValid
    ? 'crc_failure' : 'format_failure';
  if (reason === 'crc_failure') this.counters.crcFailures++;
  else this.counters.formatFailures++;
  this.awaitingReacquisition = true;
  this.candidateAlignment = null;
  this.discard(1);
  this.transition('HUNT', reason, decoded);
}
```

A.3 Experimental baseline

The control parser uses persistent BitBuffer storage and the production decodeFrame function. Its essential search and acceptance procedure is reproduced below. append and discard operate on the same logical head; each input is processed in 4 KiB batches.

```
append the next input batch
repeat:
  if no candidate is held:
    offset = 0
    while offset + 32 <= availableBits and ASM does not match:
      offset += 8
    discard offset bits
    if fewer than 32 bits remain: stop this batch
    hold candidate
  if fewer than 2080 bits remain: stop this batch
  decoded = decodeFrame(extract 2048 bits after the ASM)
  if CRC and format are valid:
    emit frame; discard 2080 bits
  else:
    count rejection; discard 8 bits
  release candidate and continue
```

A.4 Commands for verification and demonstration

```
npm test
npm run typecheck
npm run build

npm run transmitter -- --mode bit-offset --offset 3
npm run transmitter -- --mode bit-flip
npm run transmitter -- --mode bit-insert
npm run transmitter -- --mode bit-delete

# Explicit target for a longer live demonstration
npm run transmitter -- --mode bit-insert --target-frame 50
```

For distinct live streams, stop the transmitter and restart the receiver before the next mode so that counters and the sequence baseline start afresh. Do not restart the receiver during a fault-recovery demonstration. The guided lab needs only the receiver and UI; its default fault target is sequence 2.

APPENDIX B ADDITIONAL EVIDENCE AND RAW RESULTS

B.1 Recorded localhost dashboard evidence

Figures 6 and 7 show the current dashboard during guided insertion and deletion demonstrations captured on 20 October 2026. The guided lab processes a deterministic six-frame stream through the production receiver engine in memory. These views illustrate marker alignment, CRC validation and recovery; they are separate from live UDP traffic and the repeated comparison executed for this report. Replay timing is illustrative and is not a throughput measurement.

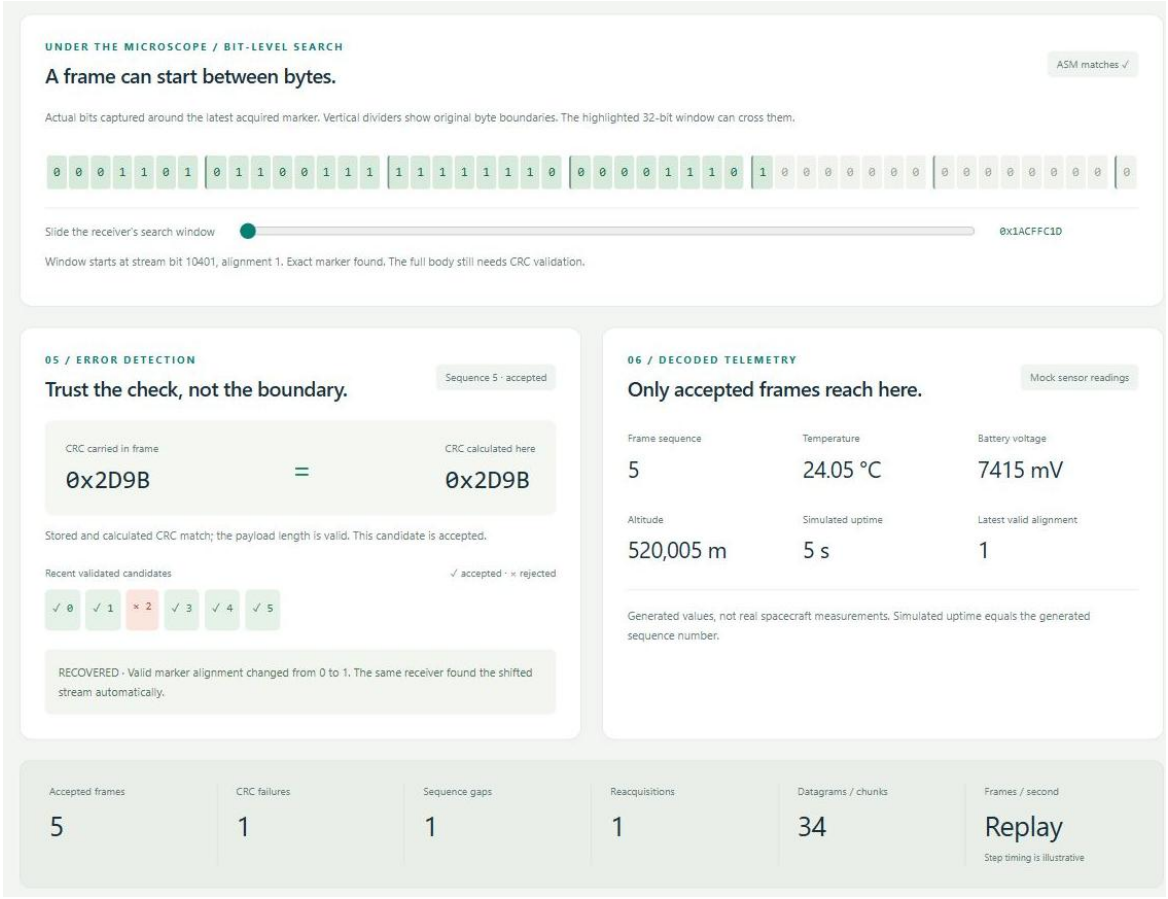


Figure 6 Guided insertion demonstration with later valid alignment 1

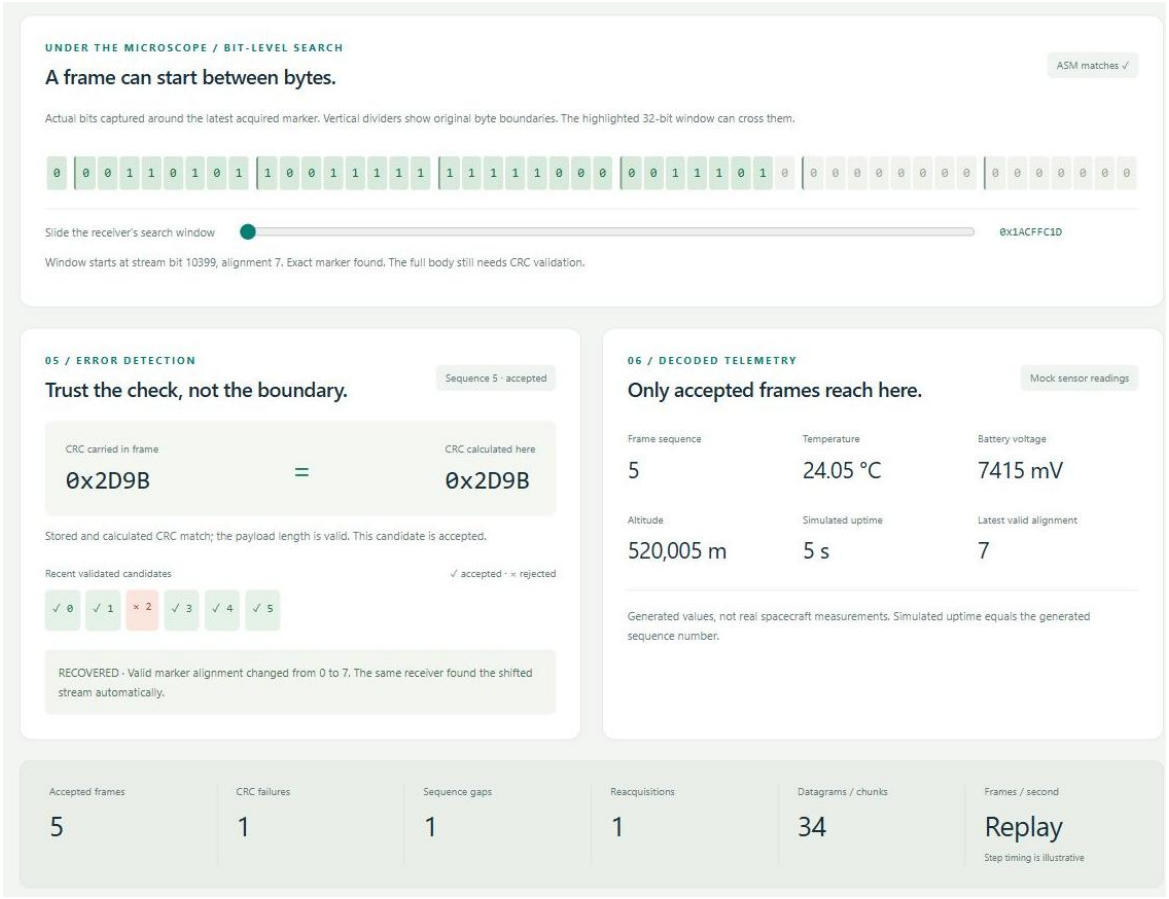


Figure 7 Guided deletion demonstration with later valid alignment 7

In both guided slip demonstrations, sequence 2 was rejected and later valid telemetry resumed. Five of the six frames were accepted. The counters showed one CRC failure, one frame-sequence gap and one reacquisition. The latest valid alignment was 1 after insertion and 7 after deletion.

B.2 Raw accepted-frame counts

Table 9 lists the count from every seed in the six main scenarios. Each entry is baseline/proposed, out of 100 generated frames. Identical outcomes across seeds produce the zero sample SD reported in Table 7.

Seed	S1	S2	S3	S4	S5	S6
42	100/100	0/100	99/99	20/99	20/99	0/100
43	100/100	0/100	99/99	20/99	20/99	0/100
44	100/100	0/100	99/99	20/99	20/99	0/100
45	100/100	0/100	99/99	20/99	20/99	0/100
46	100/100	0/100	99/99	20/99	20/99	0/100
47	100/100	0/100	99/99	20/99	20/99	0/100
48	100/100	0/100	99/99	20/99	20/99	0/100
49	100/100	0/100	99/99	20/99	20/99	0/100
50	100/100	0/100	99/99	20/99	20/99	0/100
51	100/100	0/100	99/99	20/99	20/99	0/100

Table 9 Raw accepted-frame counts for all ten paired seeds

B.3 Raw workload execution times

Table 10 records the timed workload observations in milliseconds. B and P denote the baseline and proposed parsers. The values are rounded for display; means and sample SD were calculated from unrounded measurements.

Seed	100 B	100 P	500 B	500 P	1000 B	1000 P
42	1.924	2.612	6.329	6.284	13.732	13.311
43	1.277	1.618	6.406	6.274	12.618	13.867
44	1.271	1.314	6.383	6.910	13.024	12.615
45	1.284	1.529	6.576	6.441	15.470	15.061
46	1.267	1.293	6.394	6.389	12.364	13.691
47	1.972	1.330	6.802	6.848	15.314	14.911
48	1.536	1.253	11.470	11.719	15.205	15.641
49	1.387	1.306	10.534	10.969	14.745	15.277
50	1.262	1.235	11.519	12.026	16.051	24.157
51	1.370	1.243	6.197	9.180	28.688	21.860

Table 10 Raw parser execution times for the clean workload sweep

B.4 Verification record

The repeated results in Tables 7 to 10 come from the controlled in-memory experiment. Actual UDP handling is exercised by the automated integration tests and the dated dashboard evidence. No new packet capture was collected for this evaluation.

Check	Recorded outcome
Automated suite, 20 October 2026	185 passed; 0 failed; 0 skipped
Strict TypeScript checking	Passed, exit code 0
Vite production build	Passed, exit code 0
Paired comparison	9 cases × 10 seeds × 2 parsers = 180 measured observations
Expected acceptance assertions	Passed for all 180 observations
Reference report and demonstrations	178-test and live-demo baseline dated 8 October 2026

Table 11 Verification evidence and its scope

B.5 Additional interface screenshots

Figures 8 to 10 show additional views of the current interface, captured on 21 October 2026. The guided bit-flip experiment is paused immediately after the damaged frame is rejected and the next marker is acquired.

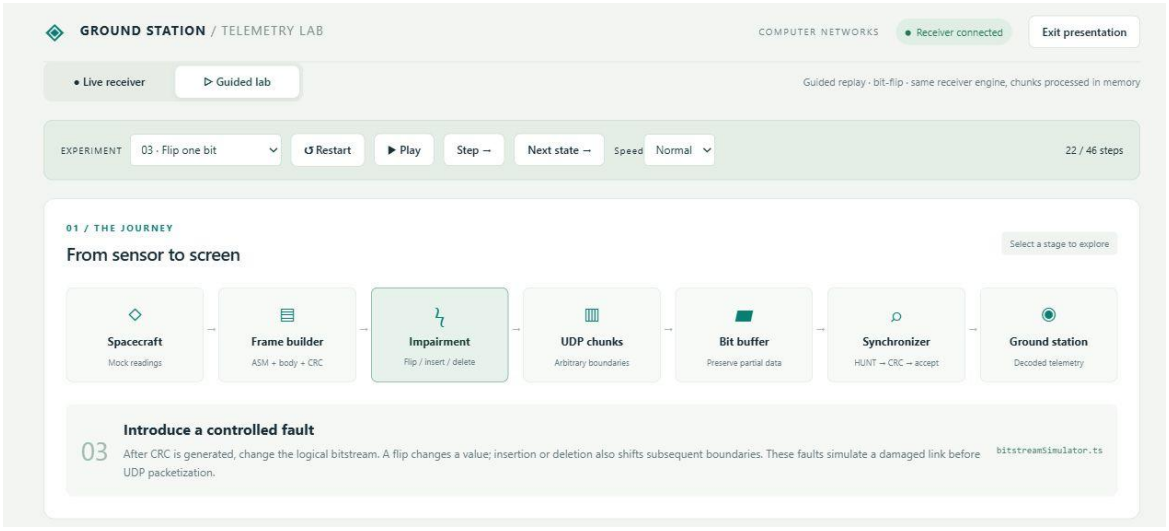


Figure 8 Transmission pipeline and guided experiment controls

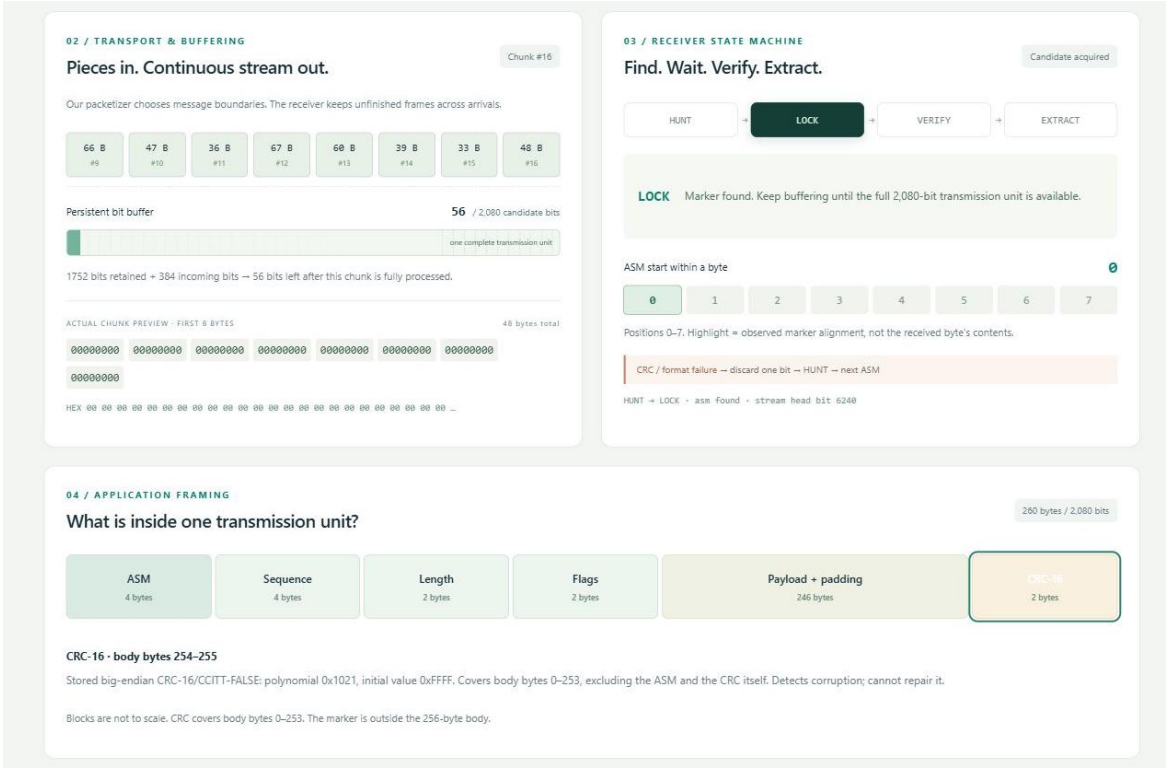


Figure 9 Persistent buffering, receiver state and frame structure

The received CRC is 0x8E09, while the receiver calculates 0x277A. Sequence 2 is rejected, and the telemetry panel retains sequence 1. The event trace records the return to HUNT and acquisition of the next marker.

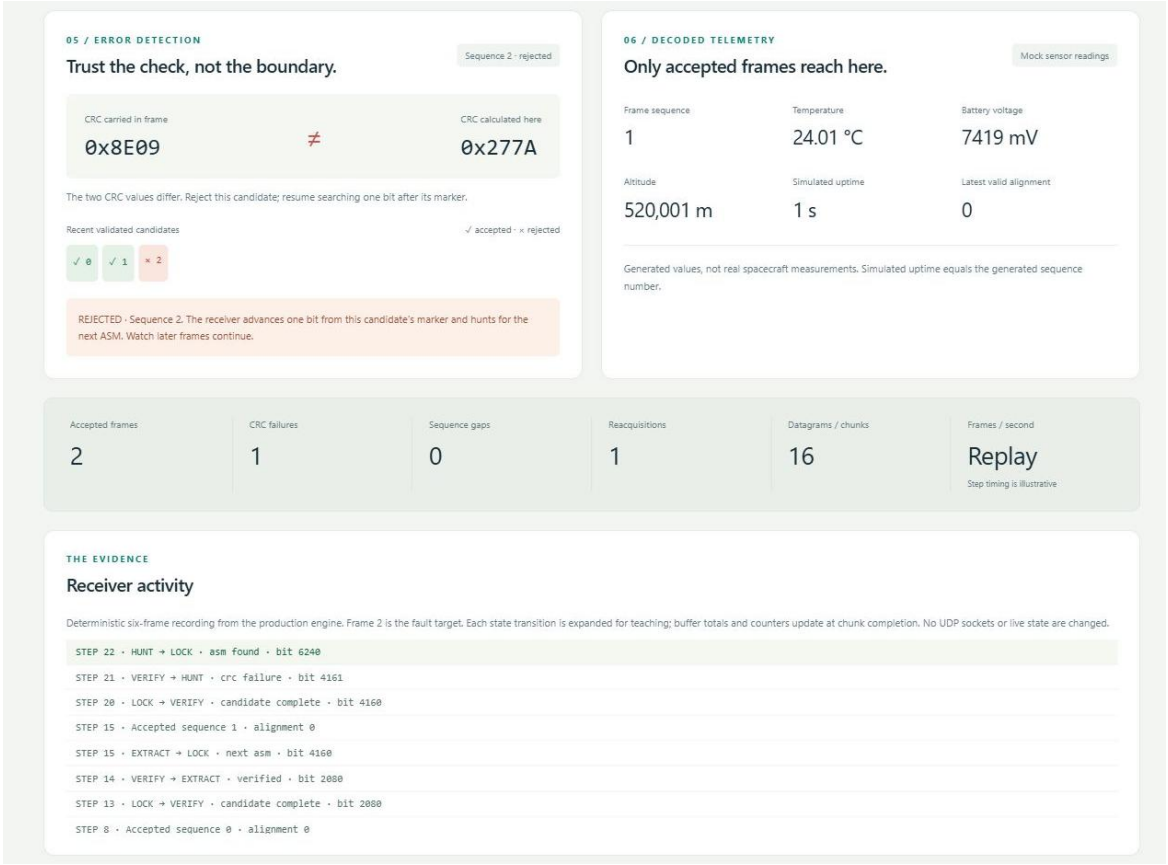


Figure 10 CRC mismatch and rejection of sequence 2